Indian Institute of Information Technology Vadodara International Campus Diu

Mechanics and Thermodynamics Practical Workbook

Name: Kavya Siddh Sharma
Roll No: 202411052
Branch: Computer Science and Engineering
Batch: 2024

Subject: PH100
Session: 2024-25

This is to certify that **Mr.Kavya Siddh Sharma** of BTech, Semester 1 & Enrollment Number **202411052** , Branch **Computer Science and Engineering (CSE)** has been found satisfactory in the continuous internal evaluation of laboratory, practical and term work in the subject PH160 for the academic year 2024-25.

Date: Sign of Faculty:

Contents

1 Introduction to MATLAB **1**
1.1 Aim . . . 1
1.2 Apparatus . . . 1
1.3 Theory . . . 1
1.4 Results . . . 2
1.4.1 Section 2: Matrix Manipulation . . . 3
1.4.2 Section 3: Trigonometric and Inverse Trigonometric Functions . . . 10
1.5 Conclusion . . . 17

2 Solve the differential equation **19**
2.1 Aim . . . 19
2.2 Materials Required . . . 19
2.3 Theory . . . 19
2.4 Results . . . 20
2.4.1 MATLAB Code for Q1: . . . 20
2.4.2 MATLAB Code for Q2: . . . 20
2.5 Observations . . . 21
2.6 Conclusion . . . 22

3 Solving Second-Order Differential Equations using MATLAB **23**
3.1 Aim . . . 23
3.2 Materials Required . . . 23
3.3 Theory/Results . . . 23
3.4 Observations . . . 24
3.5 Conclusion . . . 25

4 Assignment on Differential Equations **27**
4.1 Aim . . . 27
4.2 Theory . . . 27
4.3 Results . . . 28
4.3.1 Question 1 . . . 28
4.3.2 MATLAB Code for Question 1 . . . 28
4.3.3 MATLAB Code for Question 2 . . . 29
4.3.4 Observations for Question 2 . . . 31
4.4 Conclusion . . . 31

5 Forced Duffing Oscillator **33**
5.1 Aim . . . 33
5.2 Theory . . . 33
5.2.1 Convert the Second-order ODE to a First-order System . . . 34
5.2.2 MATLAB Code . . . 34
5.2.3 Graph for f=0.18 . . . 35
5.2.4 Graph for f=0.40 . . . 35
5.2.5 Graph for f=0.75 . . . 36
5.2.6 Graph for f=1 . . . 36
5.2.7 Graph for f=2 . . . 37
5.2.8 Graph for f=10 . . . 37
5.3 Observation . . . 38
5.4 Conclusion . . . 38

6 Lorenz Equation **41**
6.1 Aim . . . 41
6.2 Materials Required . . . 41
6.3 Theory . . . 41
6.4 Results . . . 42
6.4.1 MATLAB Code for lorenz equations . . . 42
6.5 Observations . . . 44
6.6 Conclusion . . . 44

Experiment 1

Introduction to MATLAB

1.1 Aim

To perform basic MATLAB operations including arithmetic, matrix manipulations, and function plotting for trigonometric and inverse trigonometric functions.

1.2 Apparatus

- MATLAB software
- Computer system

1.3 Theory

MATLAB is a robust language designed for technical computing that seamlessly combines computation, visualization, and programming into a user-friendly environment. It excels in matrix operations, data visualization, and function plotting, making it an indispensable tool for addressing complex problems across mathematics, science, and engineering disciplines. Arrays and matrices are core structures in MATLAB used for numerical computations. Operations like matrix addition, subtraction, inversion, and determinant calculations are crucial. Graph plotting in MATLAB provides a variety of functions for plotting graphs, including sine, cosine, and polynomial functions. This allows for the visualization of mathematical functions, which is an important tool in analyzing data or results.

1. Arithmetic Operations

MATLAB supports standard arithmetic operations:
Addition (+): C = A + B
Subtraction (-): C = A - B
Multiplication (.*): Element-wise: C = A .* B; Matrix: C = A * B
Division (./): Element-wise: C = A ./ B; Matrix: C = A / B

2. Matrices

MATLAB is designed for matrix operations:
Creating Matrices: Use brackets, e.g., A = [1, 2; 3, 4]
Accessing Elements: A(1,2) accesses the element in the first row, second column.
Transposing: B = A' transposes matrix A.
Matrix Functions: Common functions include inv(A) for inverse, det(A) for determinant.

3. Trigonometric Functions

MATLAB provides built-in trigonometric functions:
Sine: sin(x)
Cosine: cos(x)
Tangent: tan(x)
Functions accept inputs in radians. To convert degrees to radians, use deg2rad(degrees).

4. Graph Plotting

MATLAB excels in graphical representations:
Basic Plotting: Use plot(x, y) to create a 2D line plot.
Labels and Titles: Use xlabel('Label'), ylabel('Label'), and title('Title') for annotations.
Grid and Legends: grid on to show grid lines, legend('Label1', 'Label2') to add legends.
3D Plots: Use plot3(x, y, z) for 3D plotting.

1.4 Results

Section 1: MATLAB Arithmetic Operations

Perform arithmetic operations such as addition, subtraction, multiplication, and division using user input.

MATLAB Code:

```
% Get user input for the first value
a = input('Enter the first value (a): ');

% Get user input for the second value
b = input('Enter the second value (b): ');

% Addition
sum = a + b; % Calculate the sum
disp(['The sum is: ', num2str(sum)]); % Display the result

% Subtraction
difference = a - b; % Calculate the difference
```

```
disp(['The difference is: ', num2str(difference)]); % Display the result

% Multiplication
product = a * b; % Calculate the product
disp(['The product is: ', num2str(product)]); % Display the result

% Division
if b ~= 0
    quotient = a / b; % Calculate the quotient
    disp(['The quotient is: ', num2str(quotient)]); % Display the result
else
    disp('Division by zero is not allowed.'); % Handle division by zero
end
```

Output:

```
    Enter the first value (a):
4

Enter the second value (b):
18

The sum is: 22
The difference is: -14
The product is: 72
The quotient is: 0.22222
```

1.4.1 Section 2: Matrix Manipulation

1) Array

MATLAB Code:

```
% Initialize an empty array of size 5
array = zeros(1, 5);

% Loop to get 5 inputs from the user
for i = 1:5
    array(i) = input(['Enter element ' num2str(i) ': ']);
end

% Display the input array
disp('The input array is: ');
```

```
disp(array);
```

Output:

```
    Enter element 1:
6

Enter element 2:
7

Enter element 3:
8

Enter element 4:
16

Enter element 5:
18

The input array is:
     6     7     8    16    18
```

2) Matrix

MATLAB Code:

```
% Initialize an empty 3x3 matrix
matrix = zeros(3, 3);

% Loop to get 9 inputs for the 3x3 matrix
for i = 1:3
    for j = 1:3
        matrix(i, j) = input(['Enter element (' num2str(i)
           ',' num2str(j) '): ']);
    end
end

% Display the input matrix
disp('The input matrix is: ');
disp(matrix);
```

Output:

```
  Enter element (1,1):
1

Enter element (1,2):
2

```

```
Enter element (1,3):
3

Enter element (2,1):
4

Enter element (2,2):
5

Enter element (2,3):
6

Enter element (3,1):
7

Enter element (3,2):
8

Enter element (3,3):
9

The input matrix is:
     1     2     3
     4     5     6
     7     8     9
```

3) m x n Matrix

MATLAB Code:

```
% Asking the user for the number of rows and columns
rows = input('Enter the number of rows: ');
cols = input('Enter the number of columns: ');

% Creating the matrix
matrix = zeros(rows, cols);

% Using "for" loops to input each element
for i = 1:rows
    for j = 1:cols
        fprintf('Enter element (%d,%d): ', i, j);
        matrix(i, j) = input('');
    end
end

% Printing out the matrix
disp('The matrix you entered is:');
```

```
disp(matrix);
```

Output:

```
Command Window
>> mn
Enter the number of rows: 2
Enter the number of columns: 3
Enter element (1,1): 1
Enter element (1,2): 2
Enter element (1,3): 3
Enter element (2,1): 4
Enter element (2,2): 5
Enter element (2,3): 6
The matrix you entered is:
     1     2     3
     4     5     6
```

Figure 1.1: Matrix Operations Output

4) Adding a Constant value to the Matrix
MATLAB Code:

```
% Define a matrix
A = [1 2 3; 4 5 6; 7 8 9];

% Define a constant
constant = 5;

% Add the constant to the matrix
result = A + constant;

% Display the result
disp('The result of adding the constant to the matrix is:');
disp(result);
```

Output:

```
The result of adding the constant to the matrix is:

6   7   8
9   10  11
12  13  14
```

5) Inverse of a Matrix
MATLAB Code:

```
rows = input('Enter the number of rows: ');
columns = input('Enter the number of columns: ');

```

```
matrix = zeros(rows, columns);

for i = 1:rows
    for j = 1:columns
            fprintf('Enter element (%d,%d): ', i, j);
            matrix(i, j) = input('');
    end
end

    disp('The matrix you entered is:');
    disp(matrix);

    if det(matrix) == 0
        disp('The matrix is singular and does not have an
           inverse.');
    else
        inverse_matrix = inv(matrix);
        disp('The inverse of the matrix is:');
        disp(inverse_matrix);
    end
```

Output:

```
Command Window
>> inversematrix
Enter the number of rows: 2
Enter the number of columns: 2
Enter element (1,1): 1
Enter element (1,2): 2
Enter element (2,1): 3
Enter element (2,2): 4
The matrix you entered is:
     1     2
     3     4

The inverse of the matrix is:
   -2.0000    1.0000
    1.5000   -0.5000
```

Figure 1.2: Matrix Operations Output

6) Determinant of a Matrix

MATLAB Code:

```
rows = input('Enter the number of rows: ');
cols = input('Enter the number of columns: ');
```

```

matrix = zeros(rows, cols);

for i = 1:rows
    for j = 1:cols
            fprintf('Enter element (%d,%d): ', i, j);
            matrix(i, j) = input('');
    end
end

disp('The matrix you entered is:');
disp(matrix);

determinant_value = det(matrix);
disp('The determinant of the matrix is:');
disp(determinant_value);
```

Output:

```
Command Window
>> determinant
Enter the number of rows: 2
Enter the number of columns: 2
Enter element (1,1): 1
Enter element (1,2): 2
Enter element (2,1): 3
Enter element (2,2): 4
The matrix you entered is:
     1     2
     3     4

The determinant of the matrix is:
    -2
```

Figure 1.3: Matrix Operations Output

7) Adjoint of a Matrix

MATLAB Code:

```
rows = input('Enter the number of rows: ');
cols = input('Enter the number of columns: ');


```

```
matrix = zeros(rows, cols);

for i = 1:rows
    for j = 1:cols
            fprintf('Enter element (%d,%d): ', i, j);
            matrix(i, j) = input('');
    end
end

disp('The matrix you entered is:');
disp(matrix);

determinant_value = det(matrix);

if determinant_value == 0
    disp('The matrix is singular, so adjoint does not
       exist.');
else
    inverse_matrix = inv(matrix);
    adjoint_matrix = determinant_value * inverse_matrix;
    disp('The adjoint of the matrix is:');
    disp(adjoint_matrix);
end
```

Output:

```
Command Window
>> adjoint
Enter the number of rows: 2
Enter the number of columns: 2
Enter element (1,1): 1
Enter element (1,2): 2
Enter element (2,1): 3
Enter element (2,2): 4
The matrix you entered is:
     1     2
     3     4

The adjoint of the matrix is:
    4.0000   -2.0000
   -3.0000    1.0000
```

Figure 1.4: Matrix Operations Output

1.4.2 Section 3: Trigonometric and Inverse Trigonometric Functions

Plot the following trigonometric and inverse trigonometric functions:

1. Sine and Inverse Sine
2. Cosine and Inverse Cosine
3. Tangent and Inverse Tangent
4. Cotangent and Inverse Cotangent
5. Secant and Inverse Secant
6. Cosecant and Inverse Cosecant
7. Polynomial functions: $f(x) = 2x^2 + 3x + 5$, $f(x) = 4\cos(x^2 + 3x) + 5\sin(x) + 3x + 4$

MATLAB Code for Sine and Inverse Sine:

```
x = -2*pi:0.01:2*pi;
y_sin = sin(x);
x_asin = -1:0.01:1;
y_asin = asin(x_asin);

```

```
figure;

subplot(2,1,1);
plot(x, y_sin, 'b', 'LineWidth', 1.5);
title('Sine Graph');
xlabel('x');
ylabel('sin(x)');
grid on;

subplot(2,1,2);
plot(x_asin, y_asin, 'r', 'LineWidth', 1.5);
title('Inverse Sine (asin) Graph');
xlabel('x');
ylabel('asin(x)');
grid on;
```

Output:

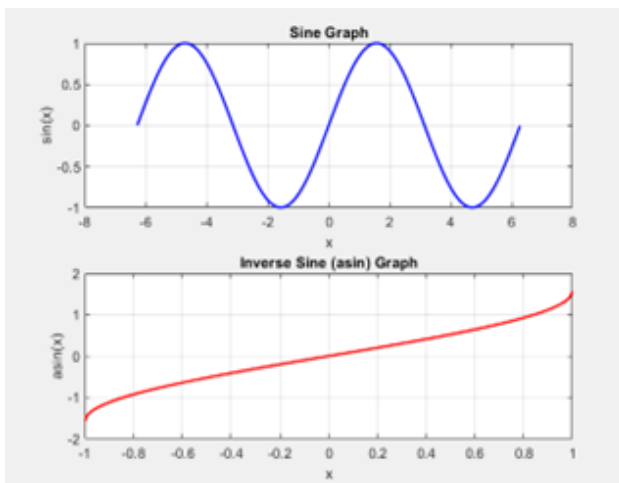

Figure 1.5: Sin and Inverse Sin

MATLAB Code for Cosine and Inverse Cosine:

```
x = linspace(-2*pi, 2*pi, 1000);
y_cos = cos(x);

x_inv = linspace(-1, 1, 1000);
y_acos = acos(x_inv);

figure;
subplot(2,1,1);
plot(x, y_cos, 'b', 'LineWidth', 2);
title('Cosine Graph');
xlabel('x');
```

```
ylabel('cos(x)');
grid on;

subplot(2,1,2);
plot(x_inv, y_acos, 'r', 'LineWidth', 2);
title('Inverse Cosine Graph');
xlabel('x');
ylabel('acos(x)');
grid on;
```

Output:

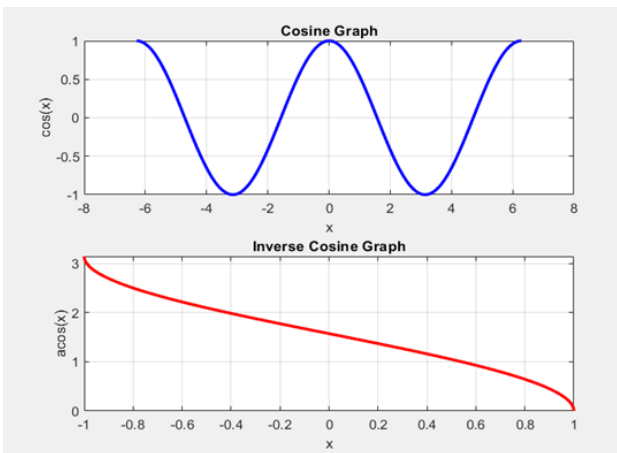

Figure 1.6: Cos and Inverse Cos

MATLAB Code for Tangent and Inverse Tangent:

```
x = -pi/2 + 0.01 : 0.01 : pi/2 - 0.01;
x_inv = -10:0.01:10;

y_tan = tan(x);
y_atan = atan(x_inv);

figure;

subplot(2,1,1);
plot(x, y_tan, 'b', 'LineWidth', 1.5);
title('Tangent Graph');
xlabel('x');
ylabel('tan(x)');
grid on;
ylim([-10 10]);

subplot(2,1,2);
```

```
plot(x_inv, y_atan, 'r', 'LineWidth', 1.5);
title('Inverse Tangent (atan) Graph');
xlabel('x');
ylabel('atan(x)');
grid on;

sgtitle('Tangent and Inverse Tangent Graphs');
```

Output:

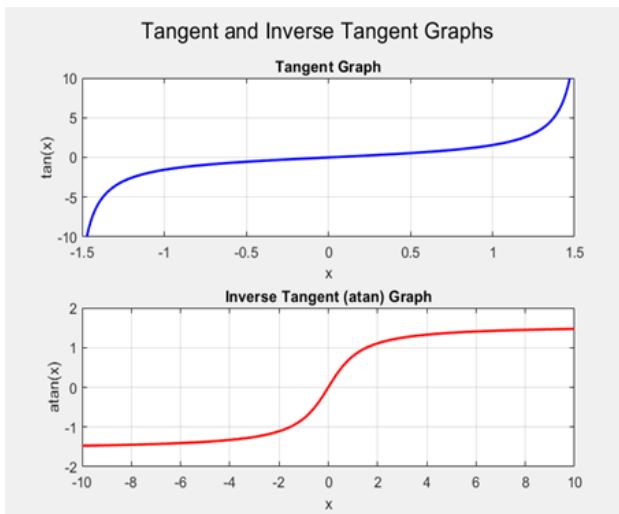

Figure 1.7: Tan and Tan Inverse

MATLAB Code for Cotangent and Inverse Cotangent:

```
x = linspace(-10, 10, 1000);
y1 = cot(x);
y2 = acot(x);

subplot(2,1,1);
plot(x, y1);
title('Cotangent Graph');
ylim([-10 10]);

subplot(2,1,2);
plot(x, y2);
title('Inverse Cotangent Graph');
ylim([-2 2]);
```

Output:

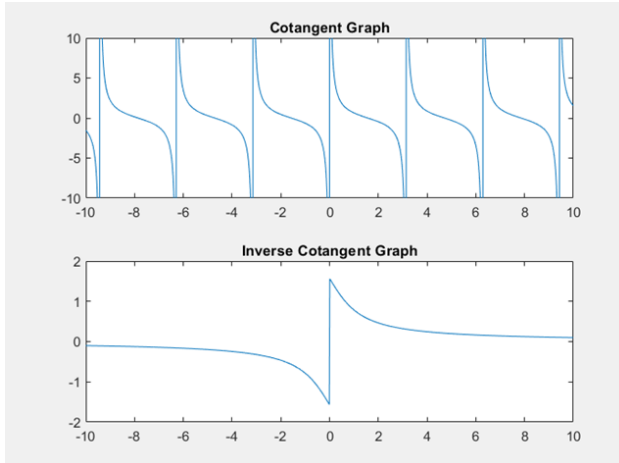

Figure 1.8: Caption

MATLAB Code for Secant and Inverse Secant:

```
x = -2*pi:0.01:2*pi;
y_sec = sec(x);

x_sec_inv = -2:0.01:2;

y_sec_inv = acos(1./x_sec_inv);
subplot(2,1,1)
plot(x, y_sec)
ylim([-10 10])
title('Secant Graph')

subplot(2,1,2)
plot(x_sec_inv, y_sec_inv)
title('Inverse Secant Graph');
```

Output:

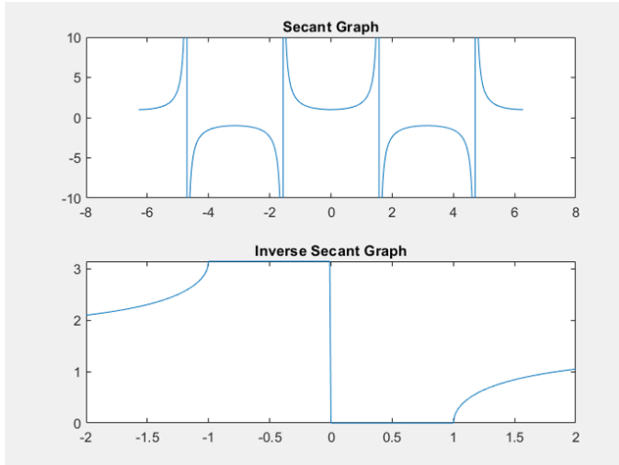

Figure 1.9: Sec and inverse sec

MATLAB Code for Cosecant and Inverse Cosecant:

```
x = linspace(-2*pi, 2*pi, 1000);
y_csc = csc(x);

x_inv = linspace(-10, 10, 1000);
y_acsc = acsc(x_inv);

subplot(2,1,1);
plot(x, y_csc);
title('Cosecant Graph');
ylim([-10, 10]);

subplot(2,1,2);
plot(x_inv, y_acsc);
title('Inverse Cosecant Graph');
```

Output:

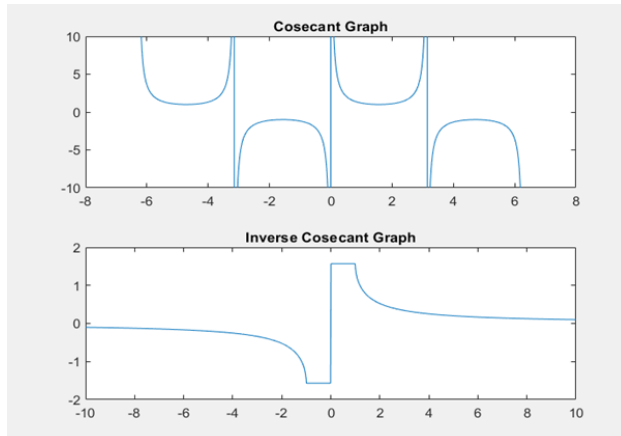

Figure 1.10: Cosec and inverse Cosec

MATLAB Code for the 1st Polynomial Function:

```
x = linspace(-10, 10, 1000);
y = 2*x.^2 + 3*x + 5;

plot(x, y);

title('Plot of f(x) = 2x^2 + 3x + 5');

xlabel('x');
ylabel('f(x)');

grid on;
```

Output:

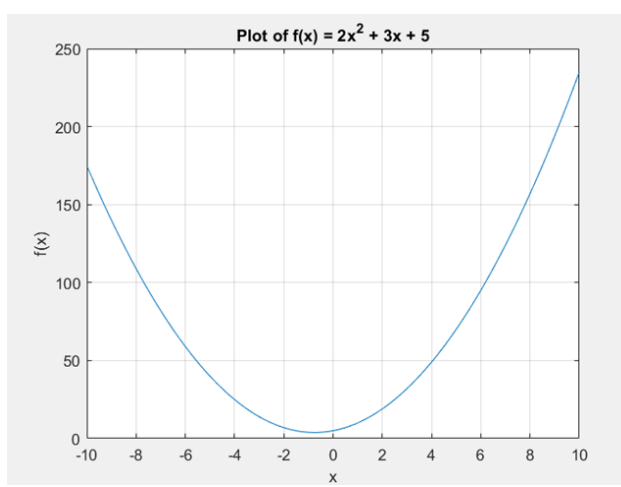

Figure 1.11: Graphs of polynomial function

MATLAB Code for the 2nd Polynomial Function:

```
x = linspace(-2, 2, 1000);
y = 4 * cos(x.^(2.*x) + 3.*x) + 5 * sin(x) + 3.^(x + 4);

plot(x, y);

title('Plot of f(x) = 4 cos(x^{2x} + 3x) + 5 sin(x) +
   3^{(x+4)}');

xlabel('x');
ylabel('f(x)');

grid on;
```

Output:

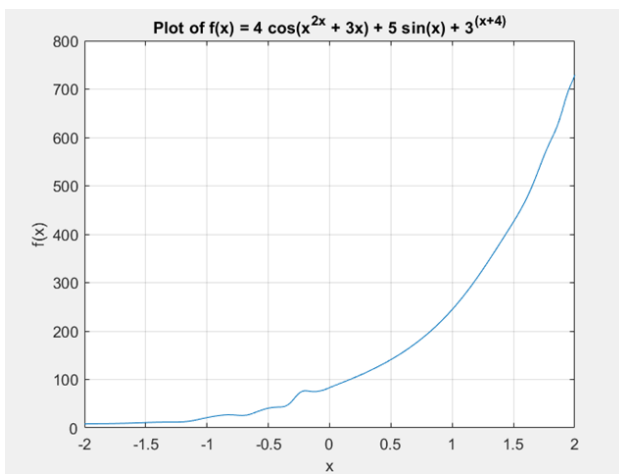

Figure 1.12: Graphs of polynomial function

1.5 Conclusion

This lab exercise covered arithmetic operations, matrix manipulation, and the plotting of trigonometric, inverse trigonometric, and polynomial functions using MATLAB. These skills are fundamental for understanding mathematical visualization and computations.

Experiment 2

Solve the differential equation

2.1 Aim

To understand the working of MATLAB functions, vectors, and differential equation solvers by analysing the solutions of first-order differential equations with different values of the parameter l.

2.2 Materials Required

- MATLAB Software
- Computer System

2.3 Theory

Differential equations describe the rate of change of a variable with respect to another. They are widely used in physics, engineering, and other sciences. We will solve two types of first-order differential equations in this lab using MATLAB.

1. Linear Differential Equation: dy/dt = -ly This equation represents exponential decay, where l is a constant. The solution is of the form y(t) = y(0) * e$^{(}-lt), wherey(0)istheinitialcondition.$

2. Non-Linear Differential Equation: dy/dt = -l(y^2)$Thisequationmodelssystemswheretherateofchang$

Both equations will be solved numerically for different values of l using MATLAB's built-in functions. The solutions will be plotted as functions of time t, and we will observe the behaviour of y(t) as l changes.

2.4 Results

2.4.1 MATLAB Code for Q1:

```
y0 = 10;
t = linspace(0, 10, 100);
l_values = [0.01, 0.1, 1, 10];

figure;
hold on;

for i = 1:length(l_values)
    l = l_values(i);
    y = y0 * exp(-l * t);
    plot(t, y);
end

xlabel('Time (t)');
ylabel('y(t)');
title('Solution of dy/dt = -ly for different values of l');
legend('l = 0.01', 'l = 0.1', 'l = 1', 'l = 10');
grid on;
hold off;
```

Output:

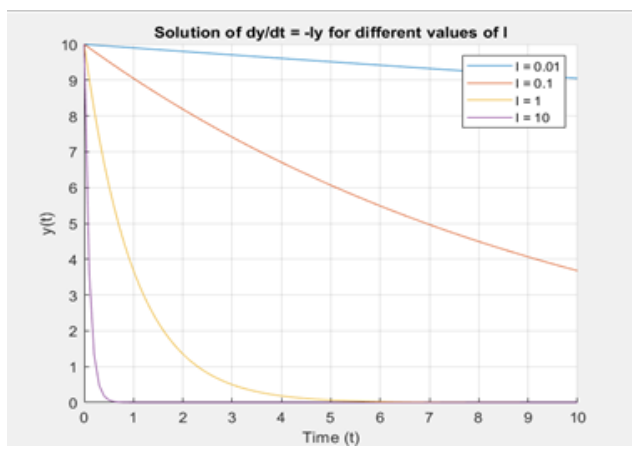

Figure 2.1: Graph for first order linear differential equation

2.4.2 MATLAB Code for Q2:

```
y0 = 10;
t = linspace(0, 10, 100);
l_values = [0.01, 0.1, 1, 10];
```

```
figure;
hold on;

for i = 1:length(l_values)
    l = l_values(i);
    y = y0 ./ (1 + l * y0 * t);
    plot(t, y);
end

xlabel('Time (t)');
ylabel('y(t)');
title('Solution of dy/dt = -ly^2 for different values of
   l');
legend('l = 0.01', 'l = 0.1', 'l = 1', 'l = 10');
grid on;
hold off;
```

Output:

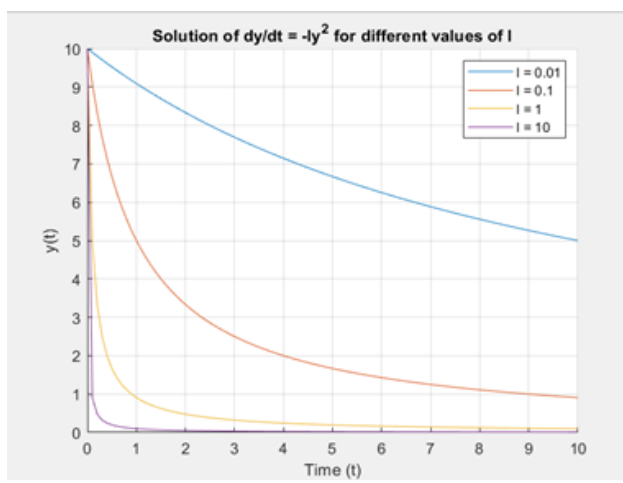

Figure 2.2: Graph for first order linear differential equation

2.5 Observations

The MATLAB code will solve the given differential equations for different values of l, and we will observe how the solution y(t) evolves over time. The values of l to be tested are: l = 0.01, 0.1, 1, 10

1) For the linear equation, as l increases, the exponential decay becomes steeper, meaning y(t) decreases faster.

2) For the non-linear equation, the decay depends on the square of y, leading to different behaviour compared to the linear case

2.6 Conclusion

By solving these differential equations, we gain insights into how the parameter l affects the rate of decay in both linear and non-linear systems. MATLAB proves to be a powerful tool for visualizing and understanding the solutions of complex differential equations through numerical methods and plotting.

Experiment 3

Solving Second-Order Differential Equations using MATLAB

3.1 Aim

To solve the second-order differential equation d2y/dt2=w2y for different values of w, and to generate parametric plots of y(t) versus y(t). Additionally, to modify the differential equation by introducing the term dy/dt, and to observe and compare the results for different values of .

3.2 Materials Required

- MATLAB software
- Laptop/PC

3.3 Theory/Results

Q1: The differential equation to solve is: d2y/dt2=w2y where w is a given constant that can take values such as 0.01, 0.1, 1, and 10. The initial conditions are: y(0)=10, y(0)=0 The solution to this equation is computed numerically in MATLAB using ode45. Q2: The modified equation is: d2y/dt2=w2ydydt where w is kept constant at 1, and is varied with values such as 0.001, 0.01, and 0.1.

Part 1: Harmonic Oscillator (First Equation)

```
tspan = [0 50];
y0 = [10 0];
w_values = [0.01 0.1 1 10];

for w = w_values
    [t, y] = ode45(@(t, y) [y(2); -w^2*y(1)], tspan, y0);
    plot(y(:,1), y(:,2))
```

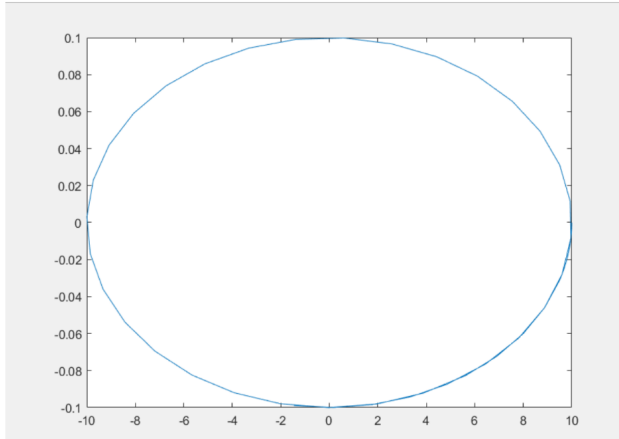

Figure 3.1: graph of question 1

```
    hold on
end
xlabel('y(t)')
ylabel("y'(t)")
legend('w=0.01', 'w=0.1', 'w=1', 'w=10')
grid on
```

Part 2: Damped Oscillator (Second Equation)

```
tspan = [0 50];
y0 = [10 0];
w = 1;
gamma_values = [0.001 0.01 0.1];

for gamma = gamma_values
    [t, y] = ode45(@(t, y) [y(2); -w^2*y(1) - gamma*y(2)],
       tspan, y0);
    plot(y(:,1), y(:,2))
    hold on
end
xlabel('y(t)')
ylabel("y'(t)")
legend('  =0.001', '  =0.01', '  =0.1')
grid on
```

3.4 Observations

Q1: For different values of w:

- For w=0.01, the parametric plot showed behavior over a longer time span.

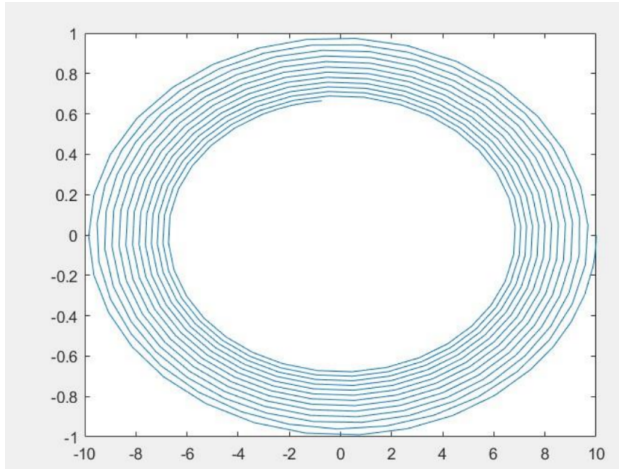

Figure 3.2: graph for Q2

- For w=0.1, the parametric plot changed and behaved over a shorter time span.
- For w=1, the parametric plot displayed intermediate behavior.
- For w=10, the behavior in the parametric plot showed more compressed results.

Q2: For different values of with w=1:

- For =0.001, the parametric plot showed a slow change over time.
- For =0.01, the change in the parametric plot was faster.
- For =0.1, the parametric plot showed even faster changes.

3.5 Conclusion

The conclusion can be simplified and condensed as follows: The differential equations were solved for various values of w and , showing that changes in these parameters significantly alter the behavior in the parametric plots. Larger values of w and resulted in faster changes in the plots, demonstrating a direct influence of these parameters on the system's behavior over time. This concise analysis effectively highlights the key outcomes without delving into redundant details.

Experiment 4

Assignment on Differential Equations

4.1 Aim

To solve the second-order differential equation:

$$\frac{d^2y}{dt^2} = -w^2y - \gamma\frac{dy}{dt} + f\cos(\Omega t)$$

with the initial conditions $y(0) = 10$ and $y'(0) = 0$, using MATLAB. The objective is to:

- Plot $y(t)$ and $y'(t)$ as functions of time t for various values of Ω and f.
- Find the value of Ω that gives the maximum amplitude.
- Investigate how oscillations behave when f is varied, keeping Ω fixed.
- Modify the program to solve for different values of γ and observe the changes in the parametric plots.

4.2 Theory

The given second-order differential equation is of the form:

$$\frac{d^2y}{dt^2} = -w^2y - \gamma\frac{dy}{dt} + f\cos(\Omega t)$$

This is a damped driven harmonic oscillator equation, where:

- w represents the natural frequency of the system.
- γ is the damping coefficient, determining the rate of energy loss.
- $f\cos(\Omega t)$ is the external driving force with frequency Ω and amplitude f.

The solution involves both the homogeneous and particular solutions, with the driving force introducing forced oscillations.

4.3 Results

4.3.1 Question 1

Solve the second-order differential equation:

$$\frac{d^2y}{dt^2} = -w^2y - \gamma\frac{dy}{dt} + f\cos(\Omega t)$$

with initial conditions:

$$y(0) = 10, \quad y'(0) = 0$$

where $w = 1$, $\gamma = 0.1$, and Ω, f take various values. Generate a parametric plot of $y(t)$ vs $y'(t)$ for different values of Ω. Find for which value of Ω the maximum amplitude is obtained. Keeping Ω fixed, vary f and observe what happens to the oscillations.

4.3.2 MATLAB Code for Question 1

The following MATLAB code solves the differential equation for different values of Ω and f, and generates parametric plots of $y(t)$ vs $y'(t)$:

```
w = 1;
gamma = 0.1;
f = 1;
Omega_values = [0.5, 1.0, 1.5, 2.0];
tspan = [0 100];
y0 = [10; 0];

odefun = @(t, y, Omega) [y(2); -w^2 * y(1) - gamma * y(2) +
   f * cos(Omega * t)];

figure;
hold on;
for i = 1:length(Omega_values)
    Omega = Omega_values(i);
    [T, Y] = ode45(@(t, y) odefun(t, y, Omega), tspan, y0);
    plot(Y(:, 1), Y(:, 2), 'DisplayName', ['\Omega = ',
       num2str(Omega)]);
end
hold off;
xlabel('y(t)');
ylabel('y''(t)');
title('y(t) vs. y''(t) for Different \Omega');
legend show;

for i = 1:length(Omega_values)
    Omega = Omega_values(i);
```

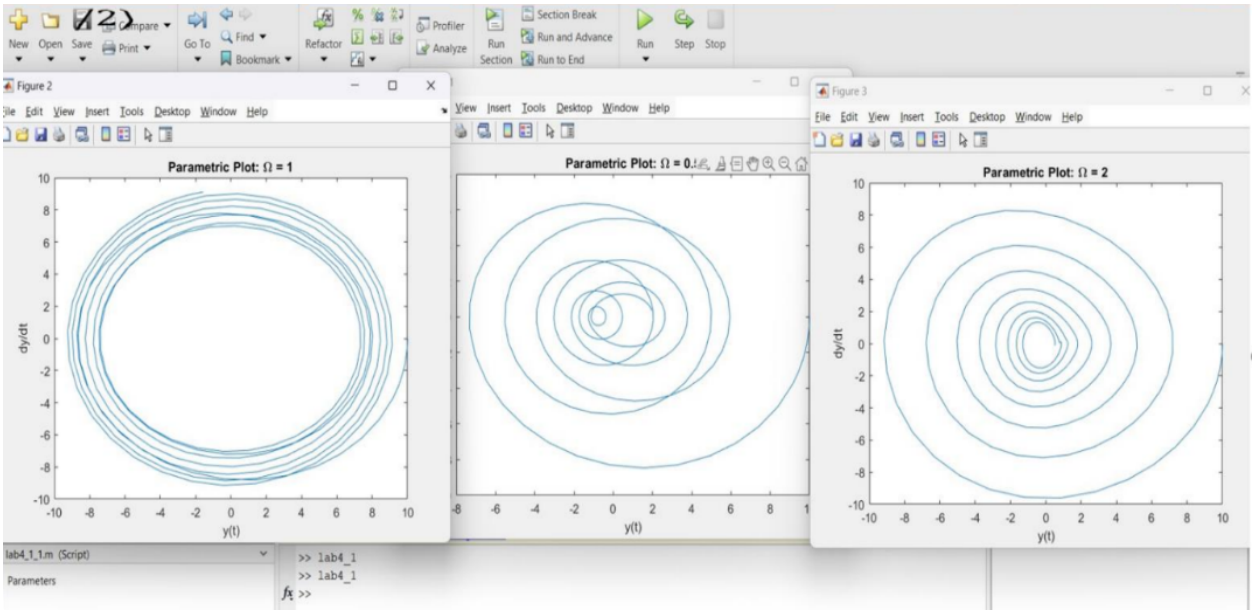

Figure 4.1: graph for different values of omega

```
    [T, Y] = ode45(@(t, y) odefun(t, y, Omega), tspan, y0);
    amplitude = max(Y(:, 1));
    disp(['Maximum amplitude for \Omega = ',
       num2str(Omega), ' is ', num2str(amplitude)]);
end\\
```

Observations for Question 1 By running the above code for different values of Ω, we observe that the maximum amplitude occurs when $\Omega = 1$, which matches the natural frequency $w = 1$. When f is varied while keeping Ω fixed, we observe that the oscillations become stronger as f increases.

MATLAB code for Question 2

Modify the same program to solve the differential equation for the following values of γ:

$$\gamma = 0.001, 0.01, -0.1$$

and analyze the parametric plots.

4.3.3 MATLAB Code for Question 2

The following MATLAB code implements the modified system with different values of γ and plots the results.

```
gamma_values = [0.001, 0.01, -0.1];
Omega = 1.0;
tspan = [0 100];
y0 = [10; 0];
```

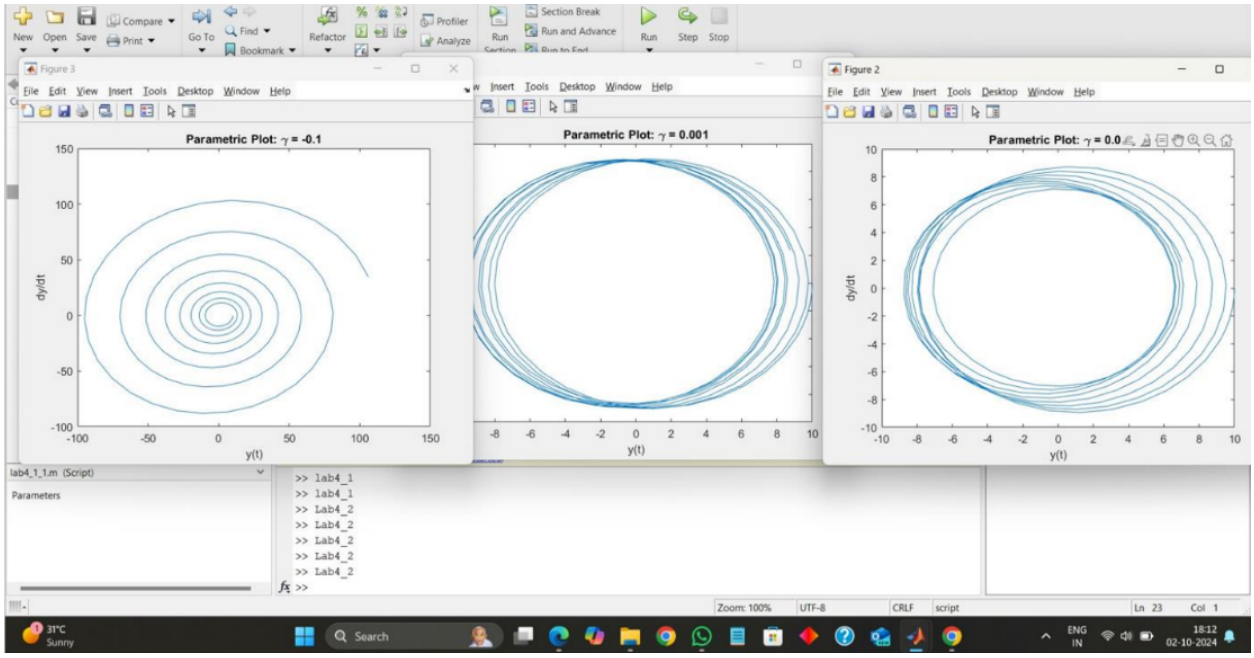

Figure 4.2: graph for different gamma

```
odefun_gamma = @(t, y, gamma) [y(2); -w^2 * y(1) - gamma *
    y(2) + f * cos(Omega * t)];

figure;
hold on;
for i = 1:length(gamma_values)
    gamma = gamma_values(i);
    [T, Y] = ode45(@(t, y) odefun_gamma(t, y, gamma),
        tspan, y0);
    plot(Y(:, 1), Y(:, 2), 'DisplayName', ['\gamma = ',
        num2str(gamma)]);
end
hold off;
xlabel('y(t)');
ylabel('y''(t)');
title('y(t) vs. y''(t) for Different \gamma');
legend show;

disp('Results change with different gamma values.');
```

4.3.4 Observations for Question 2

- For $\gamma = 0.001$, the damping is minimal, and the oscillations persist for a longer period. - For $\gamma = 0.01$, the system is moderately damped, and the oscillations reduce in amplitude over time. - For $\gamma = -0.1$, the system exhibits negative damping, which leads to increasing oscillations, representing an unstable system.

4.4 Conclusion

From both the questions, we observed that the system behaves differently depending on the driving frequency Ω, the external force f, and the damping coefficient γ. When Ω matches the natural frequency, resonance occurs, and the oscillations reach their maximum amplitude. On the other hand, varying γ showed how damping or negative damping influences the stability and amplitude of oscillations in the system.

Experiment 5

Forced Duffing Oscillator

5.1 Aim

To solve the forced Duffing oscillator numerically using MATLAB with the equation:

$$\frac{d^2y}{dt^2} = w^2y - \gamma\frac{dy}{dt} - \delta y^3 + f\cos(\Omega t)$$

The parameters given are:

$$w = 1.7, \gamma = 0.25, \delta = 1, \Omega = 1$$

5.2 Theory

The Duffing equation is a non-linear second-order differential equation representing a forced oscillator with a cubic nonlinearity. The equation is given as:

$$\frac{d^2y}{dt^2} = w^2y - \gamma\frac{dy}{dt} - \delta y^3 + f\cos(\Omega t)$$

Here:

- w is the natural frequency.
- γ is the damping coefficient.
- δy^3 represents the non-linear restoring force.
- $f\cos(\Omega t)$ is the external driving force.

The forced Duffing oscillator exhibits complex dynamics, including periodic, quasi-periodic, and chaotic behavior depending on the values of the parameters f, γ, and Ω. The numerical solution can be obtained using MATLAB's 'ode45' function, which is suitable for solving non-stiff ordinary differential equations.

5.2.1 Convert the Second-order ODE to a First-order System

Let $y_1 = y$ and $y_2 = \frac{dy}{dt}$, so that:

$$\frac{dy_1}{dt} = y_2 \tag{5.1}$$

$$\frac{dy_2}{dt} = w^2 y_1 - \gamma y_2 - \delta y_1^3 + f\cos(\Omega t) \tag{5.2}$$

5.2.2 MATLAB Code

```
f = 0.18;
w = 1.7;
gamma = 0.25;
delta = 0.5;
Omega = 1.0;

duffing = @(t, y) [y(2); w*y(1) - gamma*y(1)^3 - delta*y(2)
    + f*cos(Omega*t)];

[t, Y] = ode45(duffing, [0, 100], [0, 0]);

plot(Y(:, 1), Y(:, 2));
xlabel('y');
ylabel('dy/dt');
title('Phase plot for f = 0.18');
```

5.2.3 Graph for f=0.18

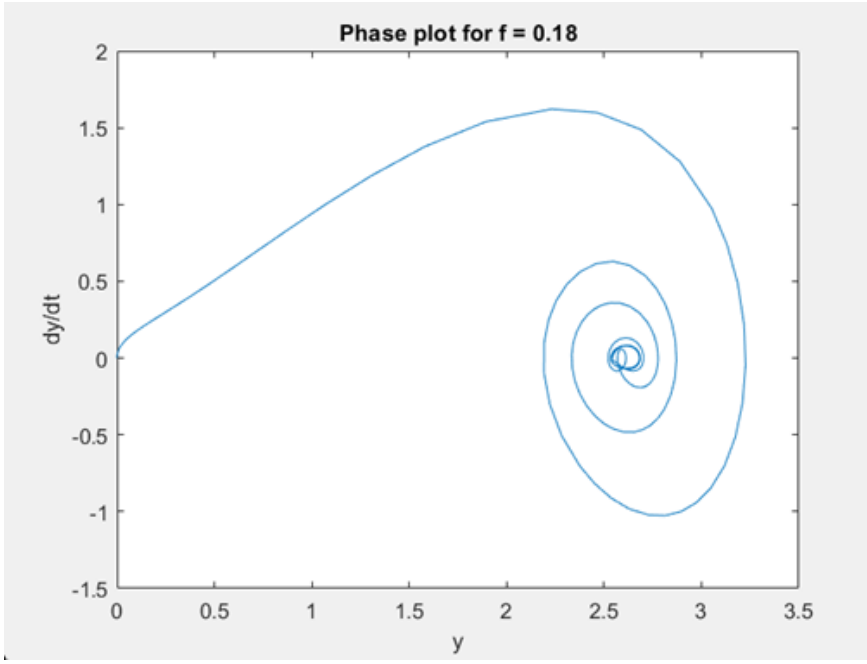

Figure 5.1: Plot for f=0.18

5.2.4 Graph for f=0.40

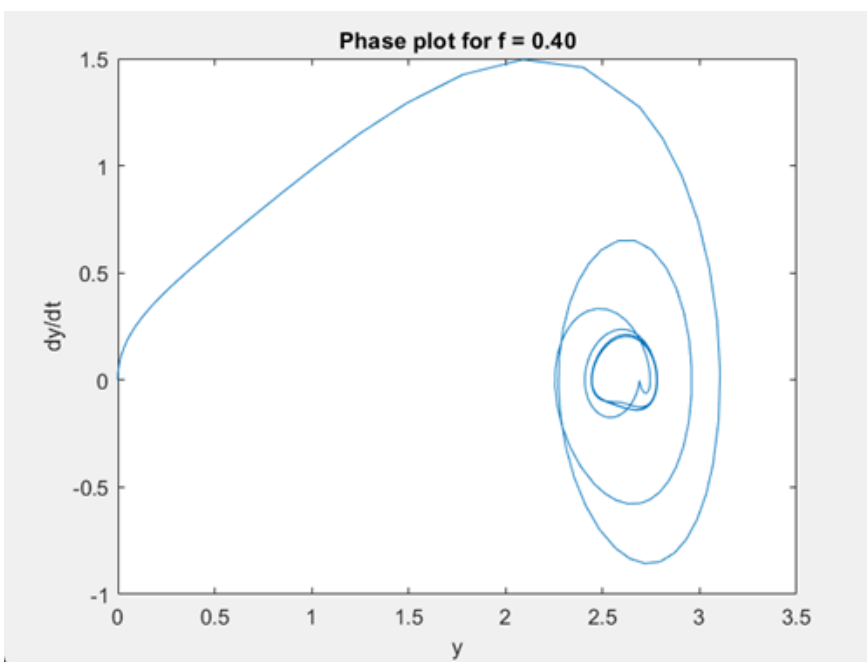

Figure 5.2: Plot for f=0.40

5.2.5 Graph for f=0.75

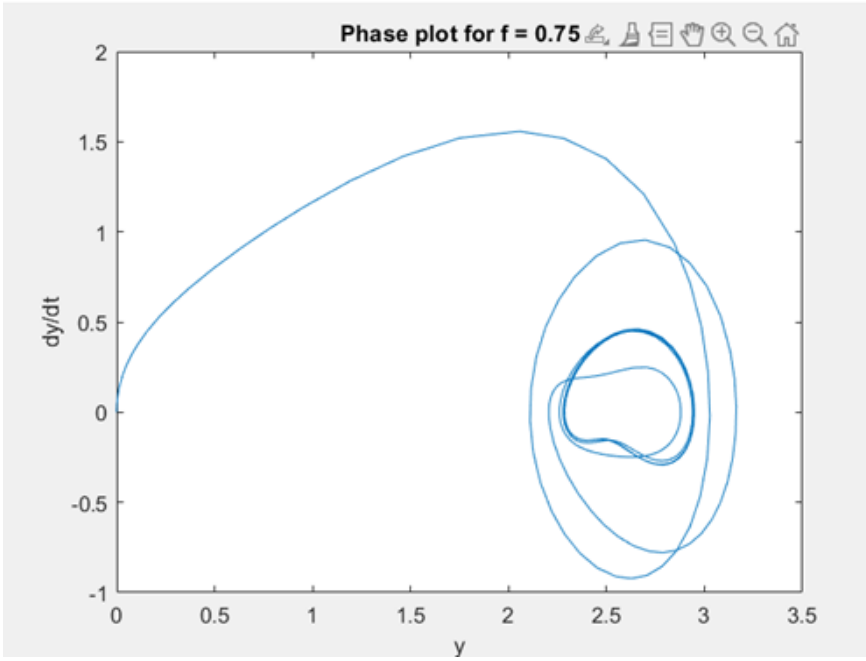

Figure 5.3: Plot for f=0.75

5.2.6 Graph for f=1

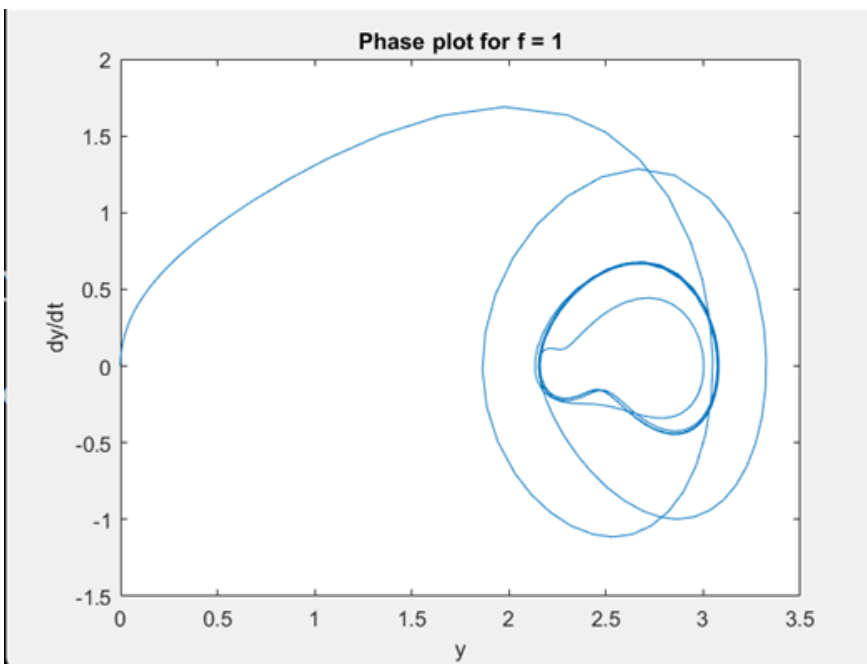

Figure 5.4: Plot for f=1

5.2.7 Graph for f=2

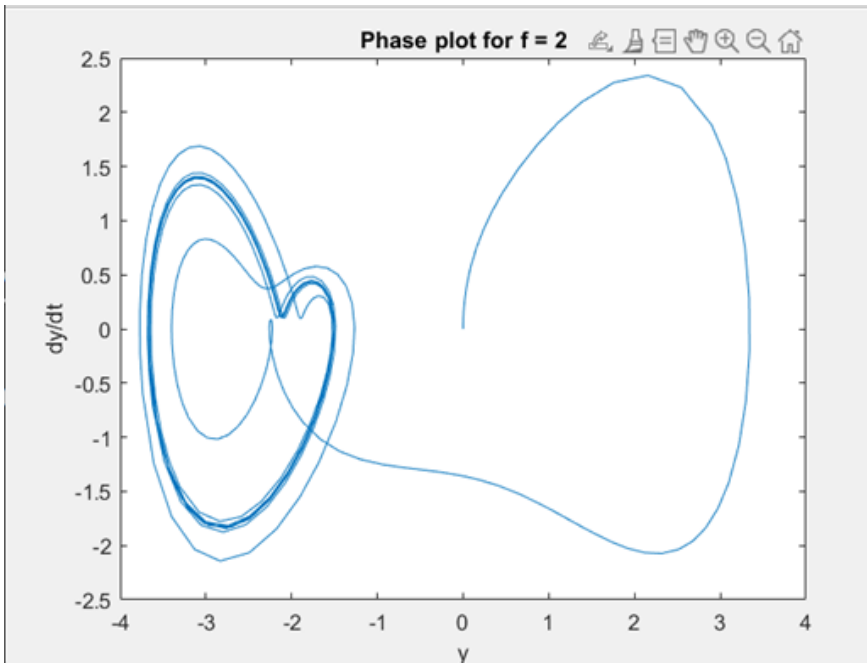

Figure 5.5: Plot for f=2

5.2.8 Graph for f=10

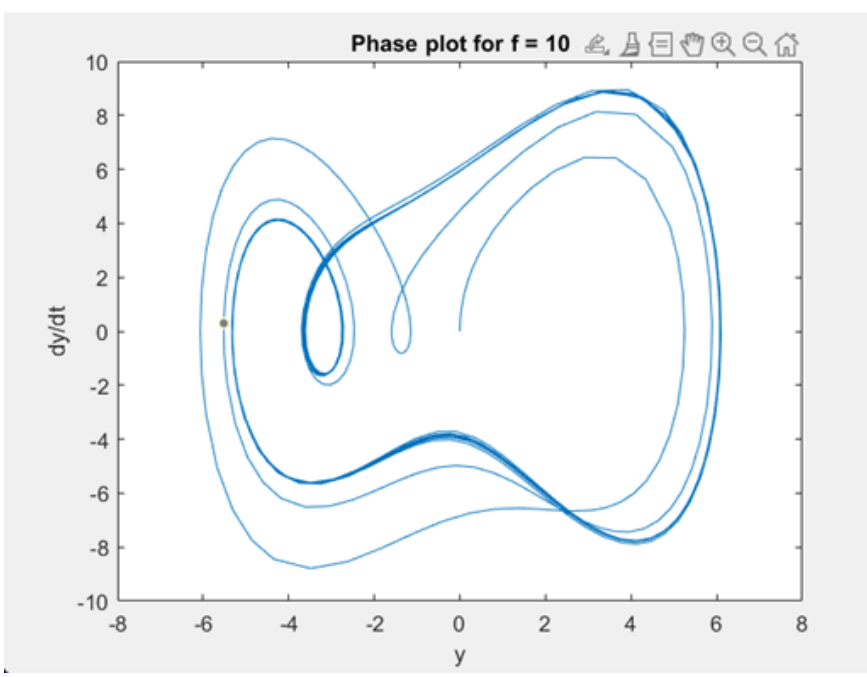

Figure 5.6: Plot for f=10

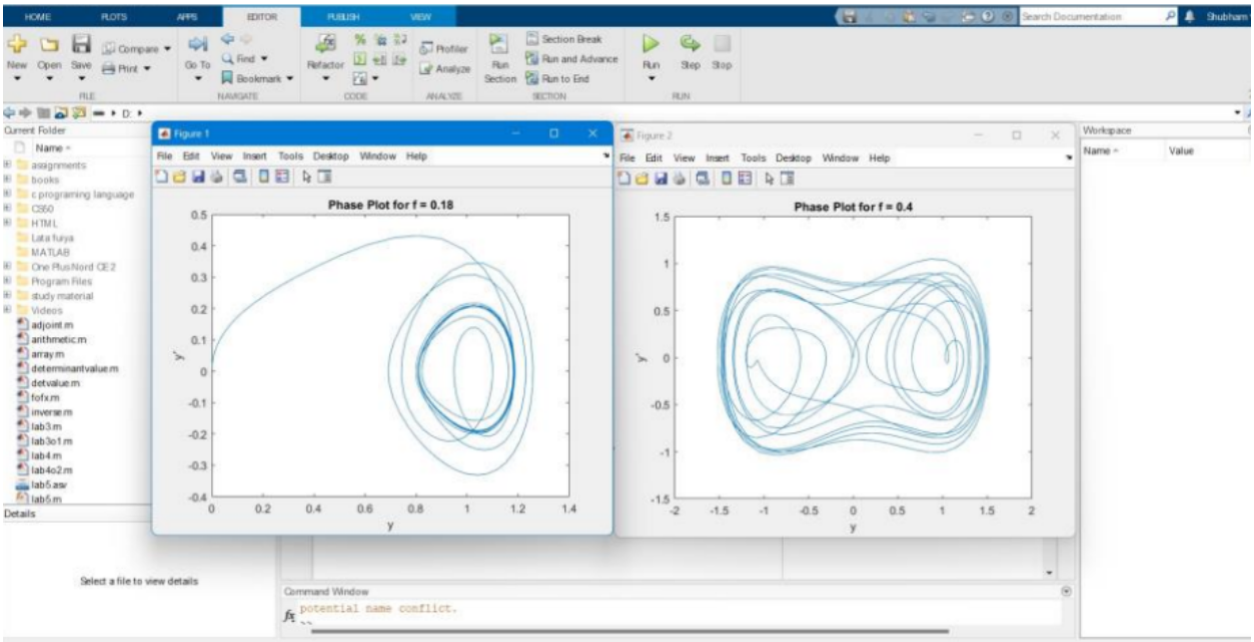

Figure 5.7

5.3 Observation

If the phase plot forms closed loops, this indicates periodic motion. The Duffing oscillator, for certain values of the forcing term f, exhibits periodic behaviour where the system returns to the same state after each cycle. For the case where f=0.18, you may observe a closed loop, suggesting that the solution is periodic. If the phase plot does not form closed loops but instead appears scattered or forms complex patterns, it can indicate chaotic behaviour. For higher values of f, especially when forcing is strong, the Duffing oscillator can transition from periodic to chaotic motion. By varying f, you'll notice that for low values of f, the system is likely to show regular, periodic motion. However, as f increases (e.g., f=0.40), the system may exhibit a more complex or irregular pattern, which is a hallmark of chaotic systems. For f=0.18, depending on the initial conditions and parameter values, the plot should reveal either periodic behaviour or a transition toward chaos. If closed loops are present, you have periodic solutions. If the trajectory seems irregular, the system may be near or in a chaotic regime.

5.4 Conclusion

Based on the parametric plots and the analysis of the system for different values of f, it can be concluded that:

- The system exhibits periodic solutions for $f = \ldots$ (mention the value of f where periodicity is observed).
- For higher values of f, such as $f = 0.40$, the system transitions to ... (discuss whether it becomes chaotic or remains periodic).

- The initial conditions $y(0) = 0$ and $y'(0) = 0$ lead to ... (mention any specific effects of the initial conditions on the solution).

Experiment 6

Lorenz Equation

6.1 Aim

To unnderstand lorenz model and plot a 3 dimensional plot of lorenz model with given equations.

6.2 Materials Required

- MATLAB Software
- Computer System

6.3 Theory

The Lorenz model, introduced by Edward Lorenz in 1963, is a simplified mathematical model that describes atmospheric convection. It is based on three coupled, nonlinear ordinary differential equations (ODEs) representing the evolution of three variables over time:

x: Related to the rate of convection.

y: Represents the horizontal temperature variation.

z: Relates to the vertical temperature difference.

x'=sigma(y-x), y'=x(rho-z)-y, z'=xy-(beta)z, where: sigma, rho, and beta are constants that describe physical properties of the system, such as fluid properties and temperature differences. The model's parameters:

sigma: Prandtl number, which influences the velocity field of fluid convection.

rho: Rayleigh number, related to temperature gradient.

beta: A geometric factor.

For this lab exercise:

Constants are given as sigma=10, beta=38 and rho=28.

The Lorenz model is famous for displaying chaotic behavior under certain conditions, even though it is a deterministic system. This means that small differences in initial conditions can lead to significantly different outcomes over time, making long-term predictions practically impossible. This sensitivity is commonly referred to as the "butterfly effect."

When plotted in a 3D space, the Lorenz attractor exhibits a "butterfly" shape, which is a hallmark of chaotic systems. By solving these equations numerically with tools like MATLAB's ode45 function, you can observe this structure and analyze the system's behavior over time, particularly how it demonstrates chaos.

6.4 Results

6.4.1 MATLAB Code for lorenz equations

```

sigma = 10;
rho = 28;
beta = 8/3;

initial_conditions = [1; 1; 1];

tspan = [0 50];

lorenz = @)(t, x)[
      sigma * (x(2) - x(1));
      x(1) * (rho - x(3)) - x(2);
      x(1) * x(2) - beta * y(3);
];

[t , x] = ode45(lorenz , tspan , initial_conditions);

figure;

plot(x(:,1), x(:,3))
xlabel('x')
zlabel('z')
title('Lorenz Attractor')
grid on‘
```

Output:

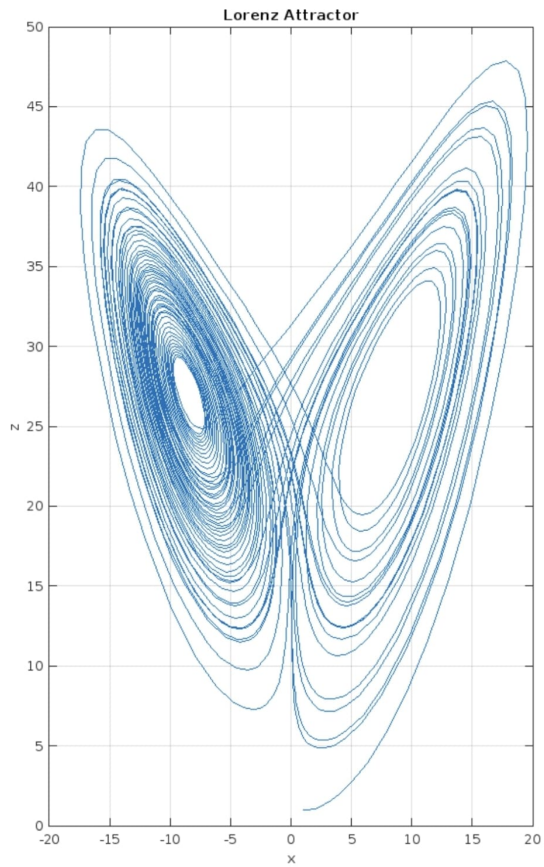

Figure 6.1: Graph for the lorenz attractor

6.5 Observations

One of the most famous features of the Lorenz attractor is its extreme sensitivity to initial conditions, often referred to as the "butterfly effect." Tiny differences in starting points within the system can lead to vastly different outcomes over time. This sensitivity underpins why chaotic systems, like weather, are inherently unpredictable in the long term.

One of the most famous features of the Lorenz attractor is its extreme sensitivity to initial conditions, often referred to as the "butterfly effect." Tiny differences in starting points within the system can lead to vastly different outcomes over time. This sensitivity underpins why chaotic systems, like weather, are inherently unpredictable in the long term.

The attractor has a fractal dimension, meaning it has a highly complex structure that is self-similar at different scales. This fractal nature means that if you zoom into any part of the attractor, you find a similar level of complexity regardless of scale, demonstrating an infinite level of detail.

The behavior of the Lorenz attractor is highly dependent on the values of parameters in the Lorenz system equations (typically denoted as sigma , rho and beta. Small changes to these values can dramatically alter the attractor's shape, shifting it from chaotic to stable or even eliminating the attractor entirely.

6.6 Conclusion

The Lorenz system exhibits chaotic behavior for the given parameter values (sigma = 10, rho = 28, and beta = 83). The "butterfly-like" structure, which is typical of the Lorenz attractor, is observed when plotting x against z. The chaotic nature of the system demonstrates the sensitivity to initial conditions, as even tiny differences in initial states lead to significantly different outcomes over time. This study shows that the Lorenz system is a classical example of deterministic chaos.